WORKING WITH INKHAVEN

Developing a story with Inkhaven

An Author's Guide to Every Kind of Book — Choosing a Track and Building It

Vladimir Ulogov

2026 · examples assume Inkhaven 1.6.10 or newer

Contents

INTRODUCTION

What kind of book are you making?

Inkhaven is one program, but it is not one tool. It holds a structured editor, a world simulation, a constructed-language workshop, a research assistant, a small family of AI readers, a citation manager, and a press that turns any of it into a finished PDF, EPUB, or manuscript. That is a great deal of machinery. The natural question — the one this book exists to answer — is not *what does each part do* (the other companion books answer that, part by part) but *which parts do I need, and in what order, for the book I am actually writing?*

Because the honest answer is: it depends on the book. A novelist and a research scientist both sit at the same desk, in the same terminal, typing into the same tree of chapters — and almost nothing else about their process is the same. The novelist grows a world and reads her prose for what it presupposes; the scientist grounds every claim in a source and checks that his citations resolve. The tools are shared. The *work* is not.

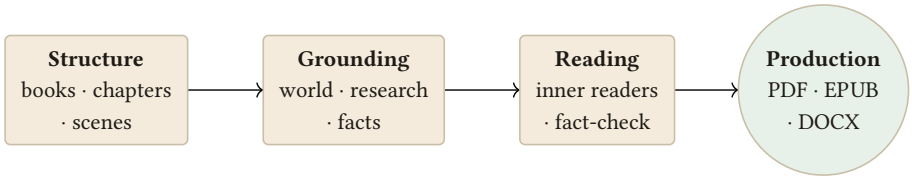
This book is about the work. It sorts the writing Inkhaven supports into a set of **tracks** — fiction, utopia, science fiction, nonfiction, scenarios, technical documentation, scientific writing, and theology or philosophy — and gives each its own full

guide: what is specific to it, which of Inkhaven's tools earn their place in it, and how those tools tie together into one working process rather than a drawer of gadgets.

> TERM **Track**
>
> A kind of literary work, and the working process suited to it. A track is not a rigid category you must belong to — it is a starting point: a recommended way to set up your project, a subset of Inkhaven's tools that pays off for that kind of book, and a rhythm of drafting and checking that fits its demands. Most real books lean on one track and borrow from a second; the guides are written so you can do exactly that.

The four faculties

However different the tracks look, each draws on the same four faculties. Every guide in this book is, underneath, an account of how one kind of book balances them.



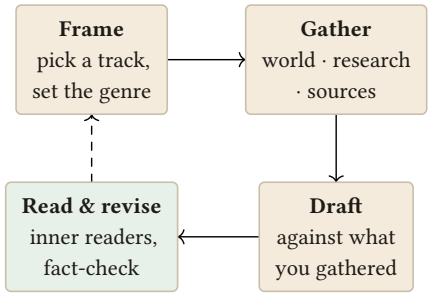
Every kind of book uses the same four faculties — a place to build structure, a way to ground it, a set of readers to test it, and a press to publish it — in proportions its track decides.

Structure is the tree your work lives in — books, chapters, scenes, and paragraphs, each a real node you can move, split, and search. **Grounding** is whatever your book must answer to so it stays consistent: a compiled world for fiction, a corpus of verified facts for nonfiction, a specification for technical writing. **Reading** is the set of second pairs of eyes Inkhaven can turn on a draft — the questioning inner readers, the fact-checker, the continuity checks. And **production** is the press: the same tree rendered into something a reader can hold.

A track is, in the end, a recipe for these four. Fiction is grounding-heavy and reading-heavy; technical documentation is structure-heavy and light on invention; scientific writing lives on grounding and citation. Learn the four, and every track is a variation you can already half-guess.

The work is a loop

None of the tracks is a straight line from blank page to finished book. Each is a loop, and the same loop:



The work is a loop, not a line: you frame the book, gather what grounds it, draft against that ground, then read and revise — returning to gather more as the work demands.

You **frame** the book — pick a track, set the genre, lay out the first chapters. You **gather** what will ground it — grow a world, research the facts, collect the sources. You **draft** against that ground rather than against a void. Then you **read and revise** — turn the inner readers and the checks on what you wrote — and return to gather more where the draft exposed a gap. The tracks differ in what they gather and how hard they check, never in the shape of the loop.

How to read this book

The first two chapters are for everyone. Chapter 1 lays the tracks side by side — what makes each distinct, and how to tell which one your book belongs to. Chapter 2 is a full tour of the desk: how to navigate and edit in Inkhaven, the companion books, snapshots, split-edit, and export — the ground every track stands on.

After that, go to your track's chapter and read it end to end; it assumes you have read the first two and nothing else. If your book straddles two tracks — a hard-SF novel is fiction and science fiction; a philosophical essay grounded in scholarship is theology and nonfiction — read both guides. They are written to combine.

> **NOTE**
>
> This is a process book, not a reference. When a chapter names a command or a keystroke, it shows enough to act; for the exhaustive treatment of any one subsystem, the sibling books go deep — **Building the World with Inkhaven** on the world simulation, **Constructed Language Development** on the ConLang suite, and **The Research Assistant** on grounding and citation. This book's job is to tell you which of them to reach for, and when.

> **INSIGHT**
>
> There is no wrong track, and no track you are locked into. The genre setting is one line in a config file; the tools are all present whatever you choose. Picking a track is not a commitment — it is a way of deciding, on day one, which of Inkhaven's many hands you actually want to hold.

Turn the page, and let us lay the tracks out.

PART I

The Shape of the Work

CHAPTER 1

1

The document tracks

Every book Inkhaven helps you write shares the same desk. What changes from one kind of book to the next is not the desk but the *work* — what you must invent, what you must not get wrong, and who you imagine reading over your shoulder. This chapter lays the eight tracks side by side so you can see, before you commit a single word, which one your book belongs to and what it will ask of you.

A track has three defining marks. First, its **ground** — the thing the book must stay true to: an invented world, a body of verified fact, a specification, a philosophical position. Second, its **risk** — the particular way this kind of book tends to fail: a novel contradicts its own map; a manual drifts from the software; a paper cites a source

that doesn't say what it claims. Third, its **reader** — the skeptic Inkhaven imagines when it reads your draft back to you, which you name with a single setting.

The one setting that colours everything

Before the tracks themselves, meet the switch that tells Inkhaven which one you are on. In your project's `inkhaven.hjson`, a single line declares the genre:

EDIT · inkhaven.hjson

```
genre: "fantasy"
```

That word is not decoration. It changes how the **AI readers** frame your prose — the Inner Socrates interrogator, the Inner Editor — so their questions are calibrated to your kind of book rather than to a generic one. Declare `theology` and the readers stop demanding empirical proof of a claim that was never meant to be empirical; declare `technical` and they read for precision and the reader's task rather than for imagery. It also seeds the right continuity categories when you set up a Facts book, and travels into the metadata of anything you export.

NOTE

The genre is free-form, and forgiving. It normalizes aliases — `sci_fi`, `science_fiction`, and `scifi` are one thing; `utopia` and `dystopian` share a frame — and an unfamiliar value simply falls back to a neutral reading rather than erroring. Leaving it unset is a valid choice: Inkhaven then judges the prose "on its own terms." Setting it is how you get a reader tuned to your track.

INSIGHT

A track is a lens, not a cage. The genre line changes framing and defaults; it never removes a tool. Everything Inkhaven can do is available on every track. The point of choosing is focus — knowing which of the many tools are *yours*.

Fiction — the invented world made consistent

The fiction track (genre `literary`, `fantasy`, `mystery`, `historical`, `romance`, `horror`, and their kin) is defined by **invention held to account**. You make everything up — and the moment you do, you take on a debt: the made-up world must not contradict itself. Its ground is the world you build; its risk is drift (the tower two different heights in chapters three and twelve, the rider crossing a continent in an afternoon); its reader is the attentive fan who remembers what you wrote four hundred pages ago.

Fiction leans hardest on **worldbuilding** and on **reading**. It is the track for which the world simulation, the character and myth layers, and the questioning inner readers were most directly built.

Utopia — the society as an argument

The utopian and dystopian track (genre `utopian`) is a special case of fiction with a philosophical spine. Its ground is not a landscape but a *premise* — "what if a society were organised like this?" — and its risk is incoherence: a designed world that quietly cheats, that declares an ideal and then relies on the very thing the ideal forbade. Its reader is the architect who asks whether the system actually holds together.

Utopia borrows fiction's whole toolkit and adds one instrument the others rarely touch: a **coherence checker** that reads the world you declared as a chain of claims and looks for the link that breaks.

Science fiction — invention that must obey a rule

The science-fiction track (genre `scifi`) is fiction with a second ground: not only must the world be self-consistent, its invented technology and physics must be *rule-bound*. The risk is the same drift as fiction plus a new one — a capability that appears when the plot needs it and vanishes when it would solve the plot too early. Its reader is the one who accepts your one impossible thing and holds you rigidly to its consequences.

Science fiction uses the world simulation like fiction, and adds the discipline of the **magic ledger** (here, a technology ledger) and a research habit for the real science it bends.

Nonfiction — the claim that must be true

The nonfiction track (genre `nonfiction`, `memoir`, `business`) inverts fiction's relationship to truth. Here you invent nothing; your ground is *the world as it is*, and your risk is the confident sentence that turns out to be wrong. Its reader is the skeptical practitioner or the newcomer who will act on what you say.

Nonfiction lives on **grounding** and **fact-checking** — the research assistant, the Facts book, the citation manager — far more than on invention.

Scenarios — the world others will play in

The scenarios track (game modules, interactive fiction, RPG sourcebooks — set up with the `rpg-sourcebook` template, genre usually `fantasy` or `scifi`) writes not a story but a *space for stories*: a place, its people, its situations, and the branches a reader-player can take. Its ground is a world that must be *usable* at speed — a game-master flipping to a location mid-session. Its risk is a gap: a place named but not described, a thread with a setup and no payoff. Its reader is the referee who needs the answer *now*.

Scenarios use the world simulation for the setting, the Places and Threads books hardest of all, and reference-grade structure so nothing is unreachable.

Technical documentation — the text that must match the system

The technical track (genre `technical`, `documentation`) documents something that *exists and changes* — software, an API, a machine. Its ground is the system itself, and its risk is staleness: prose that was true of last release and is a lie about this one. Its reader is the practitioner trying to do a task, and the newcomer who has none of your context.

Technical writing leans on **structure** and **terminology governance** — reusable blocks, a controlled glossary, reference-grade organisation — and on the AUDIENCE readers that imagine an end user rather than a fan.

Scientific writing — the argument that must be sourced

The scientific track (genre `science`, `academic`) is nonfiction at its most demanding: every load-bearing claim must trace to evidence, and the argument must survive a hostile reading. Its ground is *the literature* and *the data*; its risk is the unsupported claim and the citation that doesn't resolve. Its reader is the expert reviewer looking for the hole.

Scientific writing lives on the **research assistant**, the **Sources** book and its bibliography, and the adversarial **verdict** readers — the prosecutor who tries to break your claim before a referee does.

Theology and philosophy — the position argued in good faith

The theology-and-philosophy track (genre `theology`, `philosophy`) argues about things that are not settled by measurement. Its ground is *internal coherence* and *the tradition it answers to*; its risk is not being wrong about a fact but being incoherent, or arguing past the strongest form of the opposing view. Its reader is the theological or philosophical reader who grants the frame and presses the reasoning.

This track uses the readers most subtly of all: an Inner Theologian tuned to moral and theological weight, a Socratic interrogator that — told the genre — stops asking for proof and starts asking whether the argument holds.

How the tracks differ, at a glance

Read down this list to place your own book. The pattern is simple: as you move from fiction toward science, the balance shifts from *invention held consistent* toward *claims held true* — and the tools shift with it.

`Fiction` Ground: an invented world · Risk: self-contradiction · Reader: the attentive fan · Leans on worldbuilding + inner readers.

Utopia	Ground: a social premise · Risk: incoherence · Reader: the system architect · Adds the coherence checker.
Science fiction	Ground: world + a rule-bound novum · Risk: convenient powers · Reader: the rigorous fan · Adds a technology ledger + real-science research.
Nonfiction	Ground: the world as it is · Risk: the wrong claim · Reader: the practitioner · Leans on research + fact-check.
Scenarios	Ground: a usable world · Risk: an unreachable gap · Reader: the referee mid-session · Leans on Places, Threads, reference structure.
Technical	Ground: a changing system · Risk: staleness · Reader: the task-driven user · Leans on structure + terminology + reuse.
Scientific	Ground: the literature + data · Risk: the unsourced claim · Reader: the hostile referee · Leans on sources + adversarial readers.
Theology / philosophy	Ground: internal coherence + a tradition · Risk: incoherence, strawmanning · Reader: the good-faith opponent · Leans on the moral + Socratic readers.

ASK YOURSELF

Which sentence, if a reader caught it being false, would most embarrass your book — "the moon was full two nights running", or "the study found the opposite of what you claimed", or "this flag was removed three releases ago"? Your answer names your risk, and your risk names your track.

WHAT YOU LEARNED

- A **track** is a kind of book plus its working process, defined by three marks: its **ground** (what it must stay true to), its **risk** (how it tends to fail), and its **reader** (the skeptic Inkhaven imagines).
- The `genre` line in `inkhaven.hjson` is the one setting that tunes the AI readers, seeds continuity categories, and travels into exports — set it to get a reader calibrated to your track.
- The eight tracks run from **invention held consistent** (fiction, utopia, science fiction, scenarios) to **claims held true** (nonfiction, technical, scientific, theology/philosophy), and the tools shift along that line.
- A track is a lens, not a cage: choosing one focuses your attention on the tools that pay off, without removing any others. Most books lean on one track and borrow from a second.

PART II

The Desk

CHAPTER 2

2

The desk

Whatever your track, you will spend your hours in the same place: a terminal, a tree of chapters on the left, a paragraph under the cursor, and a set of chords that summon everything else. This chapter is the tour of that desk. Read it once, and every track guide after it will assume you can move around without being told.

A project is a tree

An Inkhaven project is not a pile of files; it is a **tree** of typed nodes. At the top sit **books** (your manuscript, and the companion books Inkhaven keeps beside it). Inside a book are **chapters**, inside chapters **subchapters**, and at the leaves **para-**

graphs — the unit you actually edit. Every node is real: you can move it, split it, rename it, search it, and give it a status.

You start a project from the command line, optionally from a template shaped for your track:

```
inkhaven init "my-book" --template novel
```

The templates — `empty`, `novel`, `nonfiction`, `technical`, `rpg-sourcebook`, and `nanowrimo` — differ only in the starting structure they lay down; you can reshape any of them freely afterward. From there, `inkhaven tui` opens the editor, and the rest of your life happens inside it.

> TERM **Node**
>
> Any element of the project tree — a book, chapter, subchapter, or paragraph — stored as a real, addressable object with its own text, status, tags, and links. Because structure is data rather than formatting, Inkhaven can reason about it: move a scene, search every paragraph by meaning, or render just the chapters you mark *ready*.

The panes, and how to move between them

The editor is a set of panes you focus with a single chord. You are always in one of them; you switch with `Ctrl` and a number or letter.

`Ctrl+1`	The Editor — the paragraph under your cursor. Where you write.
`Ctrl+T`	The Tree — the structure on the left. Navigate and reshape the book.
`Ctrl+2`	The Outline — the whole book as an indented map you can jump through.
`Ctrl+3`	The AI chat pane — a conversation grounded on your book.
`Ctrl+4 / Ctrl+/`	Search — full-text and semantic search across the project.
`Ctrl+I`	The AI prompt line — a quick instruction without leaving the editor.

Alt+← / Alt+→	Back and forward through your navigation history, like a browser.

> **NOTE**
>
> Search is not only literal. Inkhaven keeps a semantic index of your prose, so `Ctrl+/` can find the paragraph that *means* what you typed even when it shares no words with your query — invaluable in a long manuscript where you remember a scene but not its wording.

Editing a paragraph

Inside the Editor the keys are close to what your fingers already expect, with a few worth learning early:

Ctrl+S	Save. (Inkhaven also autosaves on quit — Ctrl+Q.)
Ctrl+U / Ctrl+Y	Undo / redo.
Ctrl+C / Ctrl+K / Ctrl+P	Copy / cut / paste.
Ctrl+D	Delete the current line; Ctrl+E / Ctrl+W delete to end / start of line.
Ctrl+F / Ctrl+X / Ctrl+R	Find / find-next / replace within the paragraph.
F4 / Ctrl+F4	Split-edit: open a second version of the paragraph, then accept it.
F5	Take a snapshot of the paragraph (same as Ctrl+B N).

Split-edit — revise without fear

Pressing `F4` splits the editor into two panes holding two versions of the same paragraph: the one you have, and a place to try another. You can rewrite freely in the lower pane, compare them side by side, and either discard the attempt or accept it with `Ctrl+F4`. It is the single most useful habit for revision — a way to try a bolder sentence without losing the safe one until you are sure.

Snapshots — a history you can walk back

Every paragraph keeps a history of **snapshots**. Press `F5` before a big change and Inkhaven records the current text; later you can walk back to any earlier version. This is not the same as undo — snapshots survive across sessions and give you named points to return to. Revise boldly; the earlier draft is never gone.

Status and tags — telling parts apart

A manuscript is rarely uniform: some scenes are finished, some are napkin sketches. Inkhaven lets you mark each paragraph with a **status** — from `napkin` through `first`, `second`, and `third` draft to `final` and `ready`, cycled with the `Ctrl+B r` chord. You can then export only the `ready` paragraphs, or filter the tree to see what still needs work. **Tags** (`Ctrl+B ]`) add your own labels, searchable across the book. Both matter more on some tracks than others, but every track uses them to answer "what is done, and what is left?"

The companion books

Beside your manuscript, Inkhaven keeps a shelf of **system books** — structured places for everything that is not prose but supports it. You never create them by hand; they appear when a feature needs them, and they are never exported into your finished manuscript. The ones you will meet across the tracks:

`Places`	Locations and settings. Ctrl+B p.
`Characters`	The cast — sheets, arcs, agency. Ctrl+B c.
`Notes`	Free-form working notes. Ctrl+B g.
`World`	The worldbuilding model + its checkers. Ctrl+B W.
`Facts`	Continuity facts — what must stay true. Seeded per genre.
`Research`	Grounding material — the research assistant's corpus.
`Sources`	Your bibliography — citations that render to a reference list.
`Threads`	Plot threads — setup, development, payoff. Ctrl+V Shift+H.

`Glossary`	Controlled terminology — canonical terms and banned synonyms.
`Mythology`	Declared symbols, motifs, and archetypes.
`Language`	Constructed languages — one sub-book each. Ctrl+B X.

Each is the home of a track's grounding. Which ones you live in depends entirely on what you are writing — the novelist rarely opens Sources; the scientist rarely leaves it.

The two chord families: meta and view

Almost everything beyond plain editing hangs off two prefixes. Learn the two doors, and the whole tool is a keystroke away.

`Ctrl+B` — the meta chords reach the machinery around the manuscript: the companion-book lookups above, plus `Ctrl+B B` to build the book, `Ctrl+B W` for the World overview, `Ctrl+B J` for the Inner Socrates reading, `Ctrl+B Shift+X` to fact-check the current paragraph, `Ctrl+B $` for the cost dashboard, and `Ctrl+B 0` to edit the HJSON config from inside the editor.

`Ctrl+V` — the view chords reach the readers and the connective tissue: `Ctrl+V o` for the Inner Editor overview, `Ctrl+V @` to insert a citation, `Ctrl+V a` to add a link between nodes, `Ctrl+V Shift+T` for the timeline swim-lane, `Ctrl+V Shift+R` for an editorial pass, and `Ctrl+V Space` for a searchable command palette when you have forgotten the exact chord.

> **TRY IT**
>
> You do not have to memorise any of this. Press `Ctrl+B H` for the quick-reference overlay — the full, searchable chord table, in the tool, always current. Keep it open in your first week. The chords become muscle memory faster than you expect.

> **NOTE**
>
> Every binding is rebindable. The chord tables live in a config Inkhaven reads at startup (and scripts can set with `ink.key.*`), so if a chord fights your muscle memory from another editor, change it rather than fighting it.

Getting the book out

When a draft is ready to leave the desk, one command assembles the chapters and compiles them:

```
inkhaven build
```

or `Ctrl+B B` from inside the editor. For a specific format, `inkhaven export pdf`, `inkhaven export epub`, and `inkhaven export docx` each render your manuscript — scoped, if you like, to a status (`--status ready`) or a tag, and always excluding the companion books. The `pdf` command alone carries a workshop of finishing operations — imposition, booklets, covers, watermarks, preflight — for when the book is genuinely going to press.

> **INSIGHT**
>
> The whole desk rests on one idea: your book is *structured data*, not a formatted document. Because a paragraph is a real object with a status, a history, links, and a place in a tree, Inkhaven can search it by meaning, read it back to you, check it against a world, and render just the parts you choose. Everything in the track guides that follows is an application of that one fact.

WHAT YOU LEARNED

- A project is a **tree of typed nodes** — books, chapters, subchapters, paragraphs — each real, movable, searchable, and given a status.
- You move between **panes** with `Ctrl` + a key (Editor, Tree, Outline, AI, Search) and edit with familiar keys plus **split-edit** (`F4`) and **snapshots** (`F5`) for fearless revision.
- **Companion books** (Places, Characters, World, Facts, Research, Sources, Threads, and more) hold everything that supports the prose; which you live in depends on your track.
- Two chord families reach everything: **`Ctrl+B`** (the machinery around the book) and **`Ctrl+V`** (the readers and connective tissue) — with `Ctrl+B H` as the always-current map.
- `inkhaven build` (or `Ctrl+B B`) and `inkhaven export pdf|epub|docx` turn the tree into a finished book, scoped by status or tag.

PART III

The Track Guides

CHAPTER 3

3

The fiction track

Fiction is the track Inkhaven was first built to serve, and the one that uses the widest span of its tools. You are inventing a world and a cast and a story, and the whole discipline of the track is a single promise: *nothing you invent will quietly contradict anything else you invented.* This chapter walks the working loop — frame, gather, draft, read — as a novelist runs it.

Frame — set the genre, lay the first bones

Start a project with the novel template and declare your genre, so the readers that will later question your prose are tuned to the kind of story you are telling:

```
inkhaven init "the-salt-road" --template novel
```

EDIT · inkhaven.hjson

```
genre: "fantasy"
```

The genre can be any of the fiction keys — `literary`, `fantasy`, `mystery`, `historical`, `romance`, `horror`, `ya`, `comedy` — and it changes how the Inner Socrates and Inner Editor read you later. With the project open (`inkhaven tui`), lay down the coarse structure: a few chapters, a scene or two you already hear. Don't over-plan; the point of the loop is that structure and world grow together.

Gather — build the world the story stands on

This is where fiction earns its reputation as the heaviest track. Your ground is a **world**, and Inkhaven can grow you a consistent one from a tiny definition rather than a binder of notes that drift.

The world simulation

In the World book, a small `world.hjson` describes a star, a planet, and a few choices; `inkhaven realworld compile` grows it into a full world — climate, rivers, biomes, where cities would stand — and `--materialize` writes that world into readable chapters beside your manuscript. Because the world is *computed*, the sun rises in the same place every time and a river never runs uphill.

```
inkhaven realworld compile --materialize
```

The full craft of this lives in the companion volume, *Building the World with Inkhaven*. For fiction, the point is that the world becomes *present at your desk*: `inkhaven realworld scene` gives you the season, weather, and nearest settlement for a scene at a given place and day, so you write into a real climate rather than a guessed one.

Characters, threads, and myth

The Characters book holds your cast; `inkhaven character arc` and `character check` help you declare where a character is meant to go and ask whether the pages get them there. The Threads book (`Ctrl+V Shift+H`) tracks

each plot thread from setup through payoff, so a promise made in chapter two is not forgotten in chapter twenty. And if your story leans on recurring images — a symbol that gathers weight — the Mythology book lets you *declare* them, so a later reader can check the prose keeps faith with them.

> **NOTE**
>
> A world is optional. Plenty of fiction needs only Places and Characters, not a compiled planet. Reach for the full world simulation when the setting is doing real work — when distance, season, and geography matter to the plot. A quiet domestic novel may never open the World book at all, and that is a correct use of the track.

A tongue, if the world needs one

If your peoples should speak something, the ConLang hub (`Ctrl+B X`) builds a real constructed language — phonology, lexicon, grammar, even a script — that you can translate prose into and out of. The world simulation can even propose a language per culture. This is a deep instrument with its own companion book, *Constructed Language Development*; most fiction never needs it, and the fiction that does needs it badly.

Draft — write against the world

Now the loop turns to prose. The advantage the gathering bought you is that you no longer write into a void: the world is a room you can consult. Keep a scene brief open as you write a location; drop into `Ctrl+/` to find the paragraph where you first described a character; use `F4` split-edit to try a bolder version of a line without losing the safe one.

> **INSIGHT**
>
> The discipline of fiction on this track is *write first, accept later*. The world proposes — a settlement, a name, a calendar date — and you decide what enters the manuscript. Nothing the simulation generates is written into your book without your accepting it. The author always has the last word; the tools only make the first draft of the offer.

Read — turn the questioners on the draft

When a scene is down, fiction's second heavy investment pays off: a family of AI readers that question rather than approve. None of them rewrites your prose; each asks something a good editor would.

The inner readers

Press `Ctrl+B J` for **Inner Socrates** — a classical interrogator that asks what a passage presupposes, what alternatives it closed off, what it leaves unsaid. It comes with a roster of personas you can choose among: the `first-time-reader` who notices what confuses, the `skeptical-reader` who doubts, the `careful-editor` who watches craft, the `myth-reader` who asks whether a symbol earned its weight, and the `inner-historian` who checks the page against your world's own chronology. Because you set the genre, every one of them reads *as a reader of your kind of book*.

Press `Ctrl+V o` for the **Inner Editor** — a craft reader attentive to clarity, rhythm, and momentum, tuned by genre and by your own tuning knobs.

The checks

Where the readers ask, the checks *measure*. `Ctrl+B Shift+X` fact-checks the current paragraph against the compiled world — is a journey plausible for its distance and mode, is the weather right for the date, does a claimed span of time hold? `inkhaven drift scan` watches for style and continuity drift across the book. `inkhaven realworld co-location` catches a character in two places at once. And a unified review pass (`Ctrl+B Shift+C`) folds the readers and checks into one sweep over a finished stretch.

> **PITFALL**
>
> Don't run the heavy AI readers on every keystroke. They cost money and attention, and a first draft is not the place for a skeptic. Draft a scene to its end, *then* read it — the deterministic checks (fact-check, co-location, drift) as often as you like, the LLM readers on a finished scene or chapter where a careful second pass pays for itself.

Produce — the book leaves the desk

Mark scenes with a status as they harden (`Ctrl+B r`), and when a draft is ready, `inkhaven export epub` or `inkhaven export pdf` renders it — scoped with `--status ready` if you want only the finished parts. If the book is going out on submission, the Submissions tools (`Ctrl+V u`) track agents and generate a synopsis and comparables from the manuscript itself.

> **TRY IT**
>
> Run one small loop end to end. Write a single scene set in a specific place. Open its scene brief (`inkhaven realworld scene`) and correct one detail to match the weather. Then read the scene with the `first-time-reader` persona (`Ctrl+B J`) and fact-check it (`Ctrl+B Shift+X`). You have just done, in miniature, everything the fiction track is.

Hands-on: three procedures

Here is the fiction loop as concrete keystrokes. Do these once and the abstractions above become muscle memory.

Grow a world and bring a settlement into your book

1. From the project root, create a world definition: `inkhaven realworld new "Aeloria"`. This writes a small `world.hjson` into the World book.
2. Open `world.hjson` (press `Ctrl+B W`, or edit the file) and set the star, planet, and seed to taste. Check it is well-formed: `inkhaven realworld validate`.
3. Grow the world and write it into readable chapters: `inkhaven realworld compile --materialize`. The World book now holds pages for climate, rivers, settlements, and more.
4. See what the world offers your manuscript: `inkhaven realworld proposals`. It lists settlements, a calendar, and history events waiting for your decision.
5. Accept the ones you want — `inkhaven realworld proposals accept <id>` on the command line, or

`Ctrl+B W` then `P` in the editor. Each accepted settlement becomes a Place in your Places book. Nothing enters without this step.

6. Give a hand-authored Place a spot on the map so it grounds its neighbours: `inkhaven realwo`

Write a scene against the world

1. Ask the world what a scene's setting is like: `inkhaven realworld scene --place "Harbor Town" --day 200`. It returns the season, the weather at that latitude, the biome, and the nearest named feature.
2. Draft the scene in the editor, matching what the brief told you. Use `F4` to split-edit a line you want to try two ways, and `Ctrl+F4` to accept the version you keep.
3. Check the scene holds up: `Ctrl+B Shift+X` fact-checks the paragraph against the world — travel time, weather for the date, elapsed time — and flags anything that cannot be true.

Read a finished scene

1. Open the Socratic reading: `Ctrl+B J`. Pick a persona for what you need — `first-time-reader` to find confusion, `careful-editor` for craft, `inner-historian` to test the scene against your world's chronology.
2. Open the craft reader: `Ctrl+V o` for the Inner Editor's attention to clarity and rhythm.
3. Sweep the book for style and continuity slips: `inkhaven drift scan`. Run this as often as you like — it is deterministic and cheap.

WHAT YOU LEARNED

- **Frame** with `init --template novel` and a fiction `genre`, which tunes every AI reader to your kind of story.
- **Gather** a world you can trust: `realworld compile --materialize` grows a consistent planet, Characters and Threads hold the cast and promises, Mythology declares your symbols, and the ConLang hub builds a tongue when the world needs one.
- **Draft against the world** — scene briefs, semantic search, and split-edit let you write into a real setting rather than a void; the world proposes, you accept.
- **Read** with the inner readers (`Ctrl+B J`, `Ctrl+V o`) for the questions and the checks (`Ctrl+B Shift+X`, `drift`, `co-location`) for the measurements — after a scene is drafted, not during.
- **Produce** with `export epub|pdf` scoped by status, and the Submissions tools when the book goes out.

CHAPTER 4

4

The utopia track

The utopian and dystopian track is fiction with a philosophical spine. You are still telling a story in an invented world, so everything in the fiction chapter applies — but the world here is not merely a setting. It is an *argument*: a claim that a society organised *this way* would produce *these* consequences. The discipline of the track is to make that argument hold, and Inkhaven adds one instrument the other tracks rarely touch to help you.

Frame — declare the premise, not just the place

Set the genre to the utopian frame — `utopian`, `utopia`, `dystopian`, and `dystopia` all resolve to the same reading:

EDIT · inkhaven.hjson

```
genre: "utopian"
```

This does something specific to the readers: it tells them to treat the world as a *designed system whose logic is on trial*, not as a neutral backdrop. Build the project as you would for fiction — a manuscript, Characters, Places, and a World book — but write your setting down as a set of **premises**: the founding rules of the society. This world says property is abolished; that one says memory can be edited; the other says no one may leave the city. These are the load-bearing claims, and they are what the track will hold you to.

TERM **Premise**

A declared founding rule of a designed society — the "what if" the book is built on. A premise is a claim with consequences: abolish money, and something must still allocate scarce things; edit memory, and something must decide whose. The utopia track reads your premises as a chain of such claims and looks for the link the prose quietly breaks.

Gather — the world as fiction, plus its argument

Grow the world exactly as the fiction track does — the world simulation for the physical setting, Characters for the people who live under the premise, Threads for the story that tests it. What you add is care in the World book: state the premises plainly, and state what each is *supposed* to produce. The richer that declaration, the more the coherence checker has to reason about.

Read — the coherence checker, the track's signature tool

Here is what makes utopia its own track. Inkhaven can read your declared world as a *chain of logical claims* and look for the place where the society cheats — where a premise is declared and then the prose relies on the very thing the premise forbade.

```
inkhaven world utopia-check
```

It surfaces the tensions: the abolished currency that reappears as "favours" doing exactly a currency's job; the perfect equality with an unexamined class of people who do the unpleasant work; the total surveillance the hero somehow evades without explanation. You are not obliged to resolve every one — a deliberate contradiction can be the *point* of a dystopia — so `world utopia-suppress` lets you mark a tension as intended, and `utopia-refresh` re-runs the check as the manuscript grows.

> **NOTE**
>
> The coherence findings feed the readers, too. The Inner Socrates roster includes a `utopian-architect` persona that opens each session grounded on your world's premises and its open tensions, so its questions press exactly where the argument is thinnest. With the genre set to `utopian`, even the general personas read the society as a case to be made rather than a place to be described.

> **INSIGHT**
>
> A utopia fails not when it is implausible but when it is *incoherent* — when it quietly assumes what it claims to have removed. The coherence checker exists because that failure is almost invisible from inside the draft: you declared the premise so long ago that you have stopped noticing you keep leaning on its opposite. The tool remembers what you declared and holds you to it.

Draft, revise, produce

The rest of the loop is fiction's. Draft the story that lives under the premise; read it with the questioners; and when a stretch is finished, run `utopia-check` over it to see whether the argument still holds where the new prose landed. Produce with `export epub|pdf` as any novel would.

> **PITFALL**
>
> Don't let the argument eat the story. A utopia is still fiction: readers come for people, not for a treatise. Use the coherence checker to keep the world honest, but keep the inner readers (`Ctrl+B J`, `Ctrl+V o`) turned on the *prose* — is the scene alive, does the character want something — with the same seriousness. The best books on this track are arguments you feel as stories.

Hands-on: declare a premise and put it on trial

1. Set up the world as any fiction (`inkhaven realworld new`, edit `world.hjson`, `inkhaven realworld compile --materialize`).
2. In the World book, write your founding rules as plain declarations — one premise per statement: *"No one owns land."* *"Memory may be edited by the state."* The clearer and more consequential each is, the more the checker has to reason about.
3. Run the coherence check: `inkhaven world utopia-check`. It reads your premises as a chain of claims and reports the tensions — the place where the abolished thing quietly returns.
4. Read each finding and decide. If a tension is a genuine slip, fix the prose. If it is deliberate — the crack that *is* the story of a dystopia — mark it intended: `inkhaven world utopia-suppress <id>`.
5. As you draft new chapters, re-run the check over the grown manuscript: `inkhaven world utopia-refresh`. New prose can lean on an old premise's opposite without your noticing; this catches it.
6. Turn the architect's questions on the argument: `Ctrl+B J`, then choose the `utopian-architect` persona. It opens each session grounded on your premises and open tensions, and asks where the design is thinnest.

WHAT YOU LEARNED

- Utopia is **fiction with an argument**: set `genre: "utopian"` and write your setting as a set of **premises** — the founding rules the book is built to test.
- Gather the world as any fiction would (world simulation, Characters, Threads), but state each premise and what it is meant to produce, plainly, in the World book.
- The track's signature tool is `world utopia-check` — it reads the premises as a chain of claims and finds where the society **cheats**; `utopia-suppress` marks an intended contradiction, `utopia-refresh` re-checks as you grow.
- The `utopian-architect` persona grounds its Socratic questions on your premises and open tensions — but keep the ordinary inner readers on the prose, so the argument stays a story.

CHAPTER 5

5

The science-fiction track

Science fiction is fiction that has signed a contract. You are granted one impossible thing — a faster-than-light drive, a machine that reads minds, a world without death — and in exchange you agree to obey its consequences with a straight face. The track's discipline is *rule-bound invention*: the novum works the same way on page four hundred as on page four, and the story never gets a capability it did not establish. This chapter adds to the fiction loop the two things that keep that contract: a ledger of your declared rules, and a research habit for the real science you are bending.

Frame — set the genre, name the novum

Set the genre so the readers hold you to consequence rather than to imagery:

EDIT · inkhaven.hjson

```
genre: "scifi"
```

`sci_fi` and `science_fiction` resolve to the same frame. Then, before you draft, name your novum in a sentence and, crucially, name its *rule*: not "there is faster-than-light travel" but "travel is instantaneous but only between beacons that took a decade to build." The rule is what makes the story science fiction rather than fantasy in a spacesuit — and it is what the tools will help you keep.

Gather — a world, a ledger, and real science

Grow the world as the fiction track does. Two additions are specific to this track.

The technology ledger

Your novum bends physics, and the world's fact-checker knows physics — so left alone it would flag your FTL drive as an impossible journey on every page. The World book's **magic block** (here, read it as a *technology ledger*) is where you *declare the exception*: a named rule that says what it covers and whom it applies to. Once declared, the fact-checker honours it and stops warning — while still catching the journey that breaks even your *own* rule.

TERM **Technology ledger**

The declared list of a story's departures from real physics — each a named rule stating what it overrides and where it applies. Declaring an exception is not cheating; it is the opposite. It fixes the rule in one place so the fact-checker can hold every later page to it, and so a capability can never quietly appear where you never granted it.

Research for the science you bend

Hard SF earns its authority from the real science under the invention. The **Research Assistant** (`inkhaven research`) grounds that science: it can pull structured facts, scholarly papers, and web sources into a Research book, and keep each with its provenance, so the orbital mechanics your plot leans on are right where they can be and clearly invented where they can't. When you set up a Facts book with `facts init --genre scifi`, Inkhaven seeds the continuity categories a science-fiction manuscript most often contradicts itself on — technology, physics and travel, the polity — so the book watches the right things.

> **NOTE**
>
> The two grounds pull in opposite directions, and that is the craft of the track. The technology ledger says "here is where I leave reality, on purpose"; the research habit says "everywhere else, I stayed." Keeping the line between them sharp — declared where you invent, sourced where you don't — is what a rigorous reader is really checking.

Draft and read — held to your own rule

Draft against the world as any novelist would. When you read the draft back, the science-fiction difference is in what the checks enforce. `Ctrl+B Shift+X` fact-checks a passage against the world *and* the ledger: a journey that fits your declared drive passes silently; one that exceeds even your own rule is flagged. The inner readers, told the genre, ask the science-fiction questions — the `inner-historian` persona in particular presses whether an event's timing fits the technology you established, and whether a capability used here was ever granted.

> **PITFALL**
>
> The classic failure of the track is the *convenient power*: the drive that is slow all book until the escape needs it fast, the AI that can't do the thing until the plot requires it. Declare the rule fully in the ledger *before* you need to break it, and let the fact-checker catch you leaning on the exception. A power the reader can predict is a power they can be surprised by honestly; one that bends to the plot is a cheat they can feel.

Produce

Nothing special: `export epub|pdf`, scoped by status, and the Submissions tools if the book is going out. The rigour was all upstream — in the ledger you kept and the sources you grounded — so the finished book carries its authority quietly.

Hands-on: two procedures

Declare a technology exception and hold the story to it

1. In `world.hjson`, add a `magic` block (read it as your technology ledger) that names the exception and its limit — for example a `travel_time` rule stating that beacon-to-beacon jumps are instantaneous but nothing else is.
2. Recompile so the world knows the rule: `inkhaven realworld compile`.
3. Write a passage where a ship jumps between two beacons, then fact-check it: `Ctrl+B Shift+X`. Because the exception is declared, the impossible speed passes silently.
4. Now write a passage where the ship crosses open space faster than your own rule allows, and fact-check again. This time it is flagged — the ledger honours your exception but still catches a page that breaks your *own* law.

Ground the real science

1. Open the Research Assistant: `inkhaven research`.
2. Pull the real work behind your invention: `/openalex orbital mechanics of Lagrange points` (or `/arxiv …` for a preprint). The assistant files the paper's citation to your Sources book automatically.
3. Keep the fact you need: `/fact A Lagrange point is a position of gravitational equilibrium.` — it crosses the confirmation gate and lands in your Facts book with its provenance.
4. Seed the continuity categories a science-fiction manuscript most often trips on: `inkhaven facts init --genre scifi`, then `inkhaven facts scan` to check your prose against them.

WHAT YOU LEARNED

- Science fiction is **rule-bound fiction**: set `genre: "scifi"` and, before drafting, name your novum *and its rule* — the constraint that makes it SF rather than fantasy.
- Declare each departure from physics in the **technology ledger** (the World book's magic block) so the fact-checker honours your exception while still catching a page that breaks your own rule.
- Ground the real science with the **Research Assistant** and seed the right continuity categories with `facts init --genre scifi` — declared where you invent, sourced where you don't.
- Read with the genre-tuned inner readers (the `inner-historian` presses capability and timing) and fact-check against world **and** ledger; guard hardest against the convenient power.

CHAPTER 6

6

The nonfiction track

Nonfiction turns the fiction track inside out. You invent nothing; your ground is the world as it actually is, and your one obligation is that the confident sentence be true. Where the novelist builds a world and checks the prose against it, the nonfiction writer *gathers what is known* and checks each claim against that. The tools shift accordingly: away from the world simulation, toward the research assistant, the Facts book, and the fact-checker. This chapter is the loop for the writer of the guide, the history, the argument, the memoir.

Frame — set the genre and the structure

Start from the nonfiction template and declare the genre:

```
inkhaven init "a-short-history" --template nonfiction
```

EDIT · inkhaven.hjson

```
genre: "nonfiction"
```

`memoir` and `business` share this frame; the genre tells the readers to judge for clarity, argument, and the reader's understanding rather than for imagery. Then, unlike fiction, *plan first*. Nonfiction rewards a spine: use `inkhaven plan` and the Outline (`Ctrl+2`) to lay out the argument before you write it, so each chapter knows what it must establish and what it may assume.

Gather — build a corpus you can trust

This is the heart of the track. Instead of growing a world, you grow a **corpus** of verified material, and the Research Assistant is where you do it.

```
inkhaven research
```

Inside it you ask questions in plain language and keep the answers as **Facts** — and because a fact drawn from the open web or from the model is checkable, the assistant grounds each on a real source: structured data from Wikidata, places from GeoNames, scholarly papers, public-domain books, live web pages. Each fact carries its *provenance* — where it came from and how much to trust it — so months later you can still answer "how do you know?"

> **TERM Provenance**
>
> The recorded origin of a fact — its source and its rung on a trust ladder, from a deterministic computation down through a cited paper to an unverified model guess. On the nonfiction track provenance is not bookkeeping; it is the difference between a claim you can stand behind and one you merely remember reading somewhere. It travels with the fact silently and answers, at any time, the only question a reader really has.

The full craft of this — triangulating a claim across sources, upgrading a guess to a cited fact, catching a source that has gone dead — is the subject of the companion volume, *The Research Assistant with Inkhaven*. For the nonfiction track, the essential habit is simple: *keep what you learn as facts with provenance, and write from the corpus rather than from memory.*

Draft — write from the corpus

Now the Facts book earns its keep. As you draft, the assistant can synthesise a grounded overview of a topic from your kept facts alone (`/synthesize`), or turn them into an outline you write into (`/outline`) with each point backed by a cited fact and any gap marked *needs research*. You write into a structure that already knows what it can support — and you see, plainly, where it cannot yet.

Read — check the claims, not the world

The nonfiction reading pass measures a different thing than fiction's. The `inkhaven fact-check` command (or `Ctrl+B Shift+X`) audits your prose against your kept facts. Inside the assistant, `/factcheck` sweeps the whole Facts book for per-claim accuracy and for *contradictions between facts that are each fine alone*. And because a body of nonfiction lives or dies on consistent terminology, the Glossary book governs your terms — canonical words and their banned synonyms — with an overlay (`Ctrl+V z`) that flags where the manuscript drifts.

> **NOTE**
>
> The AI readers serve this track too, with a different cast. Told a nonfiction genre, the Inner Socrates roster offers the audience personas the track needs: the `skeptical-practitioner` who will act on your advice, the `domain-newcomer` who has none of your context, the `expert-reviewer` who looks for the hole, and the `end-user` trying to get something done. Read a finished chapter through the one who most resembles your real reader.

> **INSIGHT**
>
> Fiction's risk is a world that contradicts itself; nonfiction's is a *writer* who contradicts the world. The whole apparatus — provenance, the Facts book, the fact-checker, the controlled glossary — exists to make the second failure as visible as the compiled world makes the first. You are not being asked to remember everything. You are being given a place to put it so you never have to.

Produce

Mark chapters ready and `export pdf|epub|docx`. If your nonfiction carries citations — many do — the Sources book and its bibliography belong to you as much as to the scientist; the next chapters on the scientific and scholarly tracks cover that machinery in full, and everything there applies to any nonfiction that cites.

Hands-on: two procedures

From a question to a cited paragraph

1. Open the assistant: `inkhaven research`.
2. Ground a claim on a real source: `/web what was the average marching distance of a Ro`
   The answer comes cited; a `/fact` taken from it is fact-checked before it commits.
3. Keep it: at the confirmation gate, edit if needed and accept. The fact enters your Facts book carrying its provenance.

4. When you have gathered enough, build structure from the corpus: `/outline the logistics of a Roman campaign`. Each point is backed by a cited fact, and any gap is marked *needs research*.
5. Write into that outline, then check the prose: `inkhaven fact-check` (or `Ctrl+B Shift+X` on a paragraph) audits your sentences against the facts you kept.

Set up continuity for a long work

1. Seed the continuity book for your kind of nonfiction: `inkhaven facts init --genre historical` (the genre chooses the categories the book watches).
2. Find claims in your manuscript that belong in it: `inkhaven facts scan`.
3. Audit the whole book at once, including contradictions between facts that are each fine alone: `inkhaven facts check`, or `/factcheck` inside the assistant. Each fact is marked with a verdict glyph you can see in the Facts tree.

WHAT YOU LEARNED

- Nonfiction **inverts** fiction: you invent nothing, your ground is the world as it is, and your obligation is that each claim be true — so plan the argument first (`plan`, `Ctrl+2`).
- **Gather** a trustworthy corpus in the Research Assistant (`inkhaven research`): keep answers as **Facts** grounded on real sources, each carrying its **provenance**.
- **Draft from the corpus** — `/synthesize` and `/outline` build grounded structure with cited points and marked gaps — rather than from memory.
- **Read** with `fact-check` and `/factcheck` (per-claim accuracy + contradictions), govern terms with the Glossary, and read chapters through the **audience personas** that match your real reader.

CHAPTER 7

7

The scenarios track

A scenario is not a story; it is a *space for stories*. The game module, the RPG sourcebook, the interactive fiction — each hands a reader-player a place, a cast of people, a set of situations, and the branches they may take, and then gets out of the way. The discipline of the track is *usability under pressure*: a referee is flipping to your city in the middle of a session with three players waiting, and whatever they need must be there, findable, and complete. This chapter is the loop for the writer who builds worlds others will play in.

Frame — the sourcebook template

Start from the template built for it:

```
inkhaven init "the-drowned-coast" --template rpg-sourcebook
```

Set the genre to match the fiction underneath — usually `fantasy` or `scifi` — so the readers frame the prose correctly. But the structure is the point here: a scenario is *reference-organised*. Where a novel is read once front to back, a sourcebook is read a hundred times in fragments, so build it as a tree a referee can jump through — Places at the top level, each self-contained, nothing that requires reading the chapter before it.

Gather — a world that is usable, not just consistent

The world simulation serves this track well, but for a different reason than fiction's. You are less concerned that the sun rises correctly and more that the *places are real and reachable*. Grow the world, then lean on the parts that make it navigable:

- **Places** (`Ctrl+B p`) is the book you live in — every location a player might go, described enough to run cold.
- `inkhaven realworld gazetteer` produces a consolidated reference — regions, landmarks, waters, settlements — the referee's quick-look document.
- `inkhaven realworld map` renders an actual map, so the space is not only described but seen.

Situations, not plot

A scenario's equivalent of plot is *situations* — tensions set up and left for the players to resolve. The Threads book (`Ctrl+V Shift+H`) is where you track them: each thread a hook with a setup and a range of possible payoffs, rather than a single fixed outcome. Populate the Characters book with the people who drive them — the faction leader, the informant, the monster — as NPCs a referee can voice on sight.

TERM **Situation**

A hook prepared for play but not resolved by the author — a tension, a faction at odds, a secret about to surface — with its setup fixed and its outcome left open for the players. Where a novel's thread has one payoff the author chose, a scenario's has many the table will discover. The Threads book holds both kinds; on this track you deliberately leave the ending unwritten.

Read — hunt for the gap, not the contradiction

Fiction's readers ask whether the prose is alive; the scenario's overriding question is whether anything is *missing*. The failure mode of the track is the gap: a place named on the map but never described, a faction with a leader who has no motivation, a thread with a setup and no possible payoff. So the reading pass is a completeness sweep:

- `inkhaven thread doctor` checks your threads for dangling setups and unreachable payoffs — the promise a player will chase that leads nowhere.
- The Outline (`Ctrl+2`) and the tree let you scan for the named-but-empty node.
- The status marks (`Ctrl+B r`) matter more here than anywhere: at a glance, which locations are `ready` to run and which are still a stub?

NOTE

The AI readers still help, but you point them at coverage. Ask the Inner Editor whether a location description gives a referee enough to improvise from; ask the Inner Socrates what a situation *presupposes* that the text never states — the motive left implicit, the exit not mentioned. On this track a good question is usually "what would a referee need here that isn't on the page?"

> **INSIGHT**
>
> A scenario is judged the way software is judged, not the way a novel is: by whether every path a user takes leads somewhere real. That is why structure and completeness matter more than voice here, and why the tools you lean on — Places, the gazetteer, the map, the thread doctor, the status marks — are the tools of *reference*, not of narrative. Write it as a thing to be used, and it will be.

Produce

Sourcebooks are physical objects more often than novels are. The same `export pdf` that renders a manuscript carries a full finishing workshop — imposition, booklets, covers, even a barcode — under the `pdf` command, for when the module is going to a printer rather than a screen.

Hands-on: two procedures

Make a place drawable and reachable

1. Grow the world (`inkhaven realworld compile --materialize`) so its settlements exist, and accept the ones your module uses (`inkhaven realworld proposals`, then accept).
2. Give any hand-authored location a position so it appears on the map: `inkhaven realworld lon 8`
3. Produce the referee's quick-look reference: `inkhaven realworld gazetteer --output gazetteer.md` — regions, landmarks, waters, and settlements in one document.
4. Render the map itself: `inkhaven realworld map`. The space is now both described and seen.

Set up a situation and check for gaps

1. Add a hook to the Threads book (`Ctrl+V Shift+H`): a setup with a range of possible payoffs rather than a single fixed outcome.
2. Populate the people who drive it in the Characters book (`Ctrl+B c`) — the faction leader, the informant — with enough for a referee to voice on sight.

3. Hunt for the gap: `inkhaven thread doctor` reports dangling setups and unreachable payoffs — the promise a player will chase that leads nowhere.
4. Scan the tree for named-but-empty nodes with the Outline (`Ctrl+2`), and use the status marks (`Ctrl+B r`) to see at a glance which locations are `ready` to run.

WHAT YOU LEARNED

- A scenario is a **space for stories**, judged by **usability under pressure** — build it reference-organised (`rpg-sourcebook` template) so a referee can jump to any self-contained piece mid-session.
- **Gather** a navigable world: Places (`Ctrl+B p`) as the home book, `realworld gazetteer` for the quick-look, `realworld map` to see the space.
- Track **situations, not plot**, in the Threads book — hooks with fixed setups and open payoffs — and populate Characters with runnable NPCs.
- **Read for the gap**: `thread doctor` for dangling setups and unreachable payoffs, the Outline for named-but-empty nodes, and status marks for what is `ready` to run — then `export pdf` with its print-finishing workshop.

CHAPTER 8

8

The technical track

Technical documentation describes something that *exists and keeps changing* — a piece of software, an API, a machine. You invent almost nothing; the system is already there. Your obligations are three: be precise, be findable, and stay in sync with the thing you describe. The characteristic failure is not a wrong idea but *staleness* — prose that was true of last release and is now a quiet lie. This track leans hardest on structure, on controlled terminology, and on reuse, and lightest of all on invention.

Frame — the reference template and the audience

Start from the template shaped for it, and set the genre:

```
inkhaven init "widget-api-guide" --template technical
```

EDIT · inkhaven.hjson

```
genre: "technical"
```

`documentation`, `docs`, `api_docs`, and `reference` share this frame. It tells the readers to judge for precision and for the reader's task, never for imagery or voice. And because documentation is read by someone *trying to do a thing*, decide early who that someone is — the newcomer with none of your context, or the practitioner who knows the domain and wants the specific answer. The track's readers can be either, and the difference changes every sentence.

Structure — the deepest investment on this track

Nobody reads a manual front to back; they arrive by search, mid-task, needing one answer. So structure is not scaffolding here — it is the product. Build the tree so every node is *reachable and self-contained*: a task has its own page, a reference entry stands alone, nothing requires the section before it. Use the Outline (`Ctrl+2`) and `inkhaven outline` to keep the whole shape in view, and the status marks (`Ctrl+B r`) to track which pages are current and which are stubs awaiting the next release.

Reusable blocks

Documentation repeats itself — the same warning, the same setup snippet, the same parameter table — and every copy is a copy that can go stale independently. The **Snippets** book holds a reusable block once and lets every page include it, so a fix lands everywhere at once.

TERM **Snippet**

A block of content authored once in the Snippets book and referenced from many pages, rather than copied into each. On a track whose enemy is staleness, the snippet is a direct weapon: the boilerplate that appears in forty places lives in one, so correcting it is one edit, not a hunt.

Terminology — say the same thing the same way

In technical writing, a synonym is a bug. If a thing is a "widget" on one page and a "component" on another, the reader cannot tell whether they are the same. The **Glossary** book governs this: it holds your canonical terms and the synonyms you have banned, and an overlay (`inkhaven terms check`, or `Ctrl+V z` in the editor) flags where the manuscript has drifted — a "component" where the glossary says "widget."

NOTE

Terminology governance feels bureaucratic until the first time a reader files a bug that was really a naming inconsistency. `terms suggest` can propose canonical terms from the manuscript itself; `terms check` enforces them. On a large document with more than one author, this is the difference between a reference and a guessing game.

Keep the examples true — verified code blocks

The most common way documentation goes stale is the code example that no longer compiles against the current release. Inkhaven can catch that the way the fiction track catches a contradiction: by running the example. Mark a `para:code` listing's fence with `verify` and name a runner for its language, and Inkhaven will run the snippet and flag it if it fails — a stale example becomes a red mark on the exact paragraph, like any other finding.

EDIT · inkhaven.hjson

```
docs: {
  verify: {
    enabled: true
    runners: {
      rust:   "rustc --edition 2021 --crate-type lib {file} -
o /dev/null"
      python: "python -m py_compile {file}"
      bash:   "bash -n {file}"
    }
  }
}
```

Only blocks that opt in are run — the fence carries the flag:

````
```rust verify
fn greet(name: &str) -> String { format!("Hello, {name}") }
```
````

Then `Ctrl+B Shift+D` verifies the open listing, or — for the whole book, or a CI step — `inkhaven docs verify` (exits non-zero when any block fails; `--dry-run` previews the commands, `--yes` confirms execution).

PITFALL

Verification runs the commands your config names, with your privileges — so it is **opt-in twice**: nothing runs unless you enable it and name a runner, and only blocks marked `verify` are executed (illustrative or pseudo-code stays untouched). Prefer compile-only or lint-only runners (`rustc`, `py_compile`, `bash -n`) over full test runners, and never point a runner at code you haven't read. Inkhaven will not run a freshly-cloned project's examples without your explicit `--yes`.

> **INSIGHT**
>
> A verified example is the technical track's version of the fiction track's fact-check: a deterministic, zero-AI check against ground truth. The prose can still drift, but the *code* in your docs now answers to the compiler — and the compiler doesn't forget what changed last release.

Links that resolve

The other face of staleness is the broken link — a cross-reference to a section you renamed, or an external URL that has rotted away. `inkhaven docs links` checks both. Internal cross-references are checked always and cost nothing: a link to a paragraph that no longer exists is reported with its location. External URLs are opt-in behind `--external` (it makes network requests), and — like the research dead-source sweep — it is deliberately conservative: only a hard `404`/`410` or an unreachable host counts as dead, so a bot-block or a transient hiccup never cries wolf.

```
inkhaven docs links              # internal cross-references
(fast, offline)
inkhaven docs links --external   # also check http(s) URLs for
link-rot
```

Like `docs verify`, it exits non-zero when anything is broken — drop it into the same pre-release check.

Write once — variables and profiles

Two mechanisms let one source serve many outputs. **Variables** replace a token everywhere at build time: define them once and write `{{product}}` or `{{version}}` in your prose, and every export resolves them. Change the product name in one place and the whole book follows.

EDIT · inkhaven.hjson

```
docs: {
  variables: { product: "Inkhaven", version: "1.6.9" }
}
```

Profiles let one manuscript carry more than one edition. Tag a paragraph `profile:edition:enterprise` or `profile:audience:expert` (the ordinary tagging surface, `Ctrl+B ]`), then choose a slice at export:

```
inkhaven export pdf --profile edition=enterprise --profile
audience=expert
```

A paragraph tagged for a dimension is emitted only when a matching value is requested; a paragraph with no tag for that dimension is unconditional and always appears. Dimensions you do not name are left alone — so the plain `export` with no `--profile` gives you the full authoring view with every variant present.

INSIGHT

Variables and profiles are the technical author's answer to duplication. The moment you find yourself maintaining two near-identical pages — a community and an enterprise version, a beginner and an expert path — reach for a profile before you reach for copy-paste. Two copies drift; one source with two profiles cannot.

Publish it as a website

Technical documentation usually lives as a website, so Inkhaven exports one directly:

```
inkhaven export html -o site/
```

That produces a **100% self-contained** static site — one page per chapter, a sidebar table of contents, a clean reading theme, and your images copied alongside. There are no external dependencies: no CDN, no web fonts, no scripts loaded from elsewhere, so the folder opens from disk or drops onto any static host and renders

identically offline. The chrome is localised to the book's `language`, single-sourcing variables and `--profile` slices apply exactly as they do for PDF, and site-wide values (title, author, subtitle) come from an `html.hjson` file.

You do not need to write any templates — a bare `export html` renders a complete, styled book from templates embedded in the binary. When you want to customise, write the defaults out and point the exporter at your copy:

```
inkhaven export html --eject-templates my-templates/
inkhaven export html -o site/ --templates my-templates/
```

The templates split into `functional/` (the page skeleton and navigation — the machinery) and `theme/` (the stylesheet and visual partials — the look). Any file you keep overrides that one default; everything else keeps working. So a designer can restyle `theme/theme.css` without ever touching the navigation logic.

> **INSIGHT**
>
> The split between *functional* and *visual* templates is deliberate. The machinery — how the contents tree becomes a sidebar, how pages link — is the part you want to leave alone; the look is the part you want to own. Keeping them apart means you can make the site yours without inheriting the maintenance of the parts you did not change.

Read — precision, and the reader who has your context and the one who doesn't

The reading pass here is about clarity and completeness, not craft. Turn the audience personas on a finished page: the `domain-newcomer` will show you every place you assumed knowledge the reader lacks; the `end-user` will show you where the page describes the system instead of helping them *do* something; the `expert-reviewer` will find the imprecise claim. Ask the Inner Socrates what a procedure *presupposes* — the installed dependency you never mention, the state the reader must already be in — because an unstated precondition is the most common way a correct instruction fails a real user.

> **INSIGHT**
>
> Everything on this track serves one goal: that the document keep matching the system as the system moves. Snippets shrink the surface that can drift; terminology governance keeps the naming stable; status marks show what has been reviewed against the current release; reference structure means a change touches one findable page. You cannot stop the software from changing. You can make your documentation cheap to change with it.

Produce

`export pdf|epub|docx` renders the manual; scope by status so a release ships only the pages reviewed against it (`--status ready`), leaving the stubs for next time. Many technical documents also live as a website or a wiki — the structured tree and the Markdown export make that a mechanical step rather than a rewrite.

Hands-on: two procedures

Reuse a block so a fix lands everywhere

1. Open the Snippets book and author the block once — a warning, a setup step, a parameter table — as its own paragraph.
2. From any page that needs it, reference the snippet with a `#include` rather than pasting a copy.
3. When the block changes, edit it once in the Snippets book. Every page that included it now shows the correction — no hunt through forty copies.

Govern a term across the whole document

1. Let Inkhaven propose canonical terms from your own prose: `inkhaven terms suggest`.
2. In the Glossary book, record the canonical word and the synonyms you are banning — "widget", not "component" or "part".
3. Enforce it: `inkhaven terms check` reports every place the manuscript drifted from the canonical term, and the overlay `Ctrl+V z` shows the same inside the editor as you write.
4. Read a finished page through the reader who lacks your context: `Ctrl+B J`, then the `domain-newcomer` persona, to surface every assumed step; and the

`end-user` persona to catch where the page describes the system instead of helping the reader *do* something.

WHAT YOU LEARNED

- Technical writing describes a **changing system**; its enemy is **staleness**. Set `genre: "technical"` (or `documentation`) and the `technical` template, and decide whether your reader is the newcomer or the practitioner.
- **Structure is the product**: build a reachable, self-contained reference tree, and track currency with status marks.
- Fight repetition with **Snippets** (one block, many pages) and enforce naming with the **Glossary** and `terms check` (`Ctrl+V z`) — a synonym is a bug.
- **Read for precision and unstated preconditions** through the audience personas (`domain-newcomer`, `end-user`, `expert-reviewer`); ship only status-`ready` pages so the doc matches the current release.

CHAPTER 9

9

The scientific track

Scientific and scholarly writing is nonfiction at its most exacting. Every load-bearing claim must trace to evidence; the argument must survive a reader who *wants* to break it; and every citation must resolve to a source that actually says what you claim it says. Your ground is the literature and the data; your risks are the unsupported assertion and the citation that doesn't hold. This chapter adds to the nonfiction loop the two disciplines that define the track: sourcing everything, and reading adversarially.

Frame — the academic genre

Set the genre so the readers hold you to evidence and to a hostile standard:

EDIT · inkhaven.hjson

```
genre: "science"
```

`academic`, `scholarly`, and `research` share this frame; `science` and `popular_science` lean the same way with a lighter touch for a general audience. Everything in the nonfiction chapter applies — plan the argument, build a corpus, write from it — so read that chapter first. What follows is what this track adds.

Gather — the literature, with its papers named

The Research Assistant (`inkhaven research`) is the whole of this track's gathering, and here you use its scholarly reach in earnest. It can pull the top work on a question from the scholarly indexes — a paper by its DOI, a preprint from arXiv — and, crucially, *auto-file its citation* to the Sources book as it does. It can triangulate a claim across independent sources and report whether they agree, refute a claim by trying to break it, and upgrade a tentative note into a cited fact once a source is found. The companion volume, *The Research Assistant with Inkhaven*, is essentially this track's deep manual.

TERM **Triangulation**

Testing a claim against several independent sources at once and asking whether they agree — so the verdict is not "a source says so" but "the sources concur." On the scientific track it is the difference between a claim you can defend and one you found stated confidently in a single place. A claim that survives triangulation *and* a genuine attempt to refute it has earned its place in the argument.

Cite — the Sources book and the bibliography

Where fiction has a World book, the scientific track has a **Sources** book — and it is the one you never leave. Every grounded answer files a citation there automatically, and you manage the whole from the shell for interchange with the tools you already use:

```
inkhaven sources import zotero-export.bib
inkhaven sources export --format csl-json --out sources.json
inkhaven sources check
```

BibTeX comes in from a reference manager; BibTeX or CSL-JSON goes back out, closing the round-trip with Zotero and its kin. `sources check` validates every entry and exits non-zero on a problem, so it fits a continuous-integration step. Inside the editor, the cite picker (`Ctrl+V @`) drops a citation into your prose where the cursor sits, and the accumulated Sources render into a formatted reference list.

> **NOTE**
>
> A citation that no longer resolves is worse than no citation — it looks like authority and delivers nothing. The Research Assistant's dead-source check (`/deadsources`) scans your kept web sources for link-rot and flags the ones that have quietly died, so a reference does not fail under a referee's click.

Read — adversarially

Every other track's readers ask in good faith. The scientific track keeps two that do not. The **verdict** personas — the `prosecutor` and the `defender` — argue your claims rather than question them: the prosecutor tries to break a claim before a reviewer does, the defender answers. And the `expert-reviewer` audience persona reads a finished section looking for exactly the hole a referee will find. Run them on your argument *before* you submit it, when the hole is still cheap to close.

The deterministic checks matter here too. `/factcheck` sweeps the Facts book for per-claim accuracy and for contradictions between claims that are each fine alone; `/undisputed` checks the axioms you have declared for internal coherence. And for anything computed — a rate, a distance, a growth over time — the assistant's

`/calc` produces a fact whose provenance is *computed*, un-fabricatable, needing no source because arithmetic is its own authority.

> **INSIGHT**
>
> The scientific track's whole discipline is *make the referee's job easy and then do it yourself first.* Source every claim so the trail is there; triangulate and refute so the claim is strong; run the adversarial readers so the weak point is found in your office, not in review. Nothing here makes the argument for you. It makes the argument *checkable* — which, on this track, is the same as making it credible.

Produce

`export pdf|docx` renders the paper or the book with its reference list assembled from the Sources you gathered. Because every citation was filed from a real source with a real identifier, the bibliography is not a hand-typed hope that a reference looks right — it is the actual, resolvable record of what grounded the work.

Hands-on: two procedures

Cite a paper, end to end

1. In the assistant (`inkhaven research`), pull the work: `/openalex CRISPR off-target effects`. The citation is filed to your Sources book automatically.
2. Bring in a library you already curate: `inkhaven sources import zotero-export.bib`.
3. Drop a citation into your prose where the cursor sits: `Ctrl+V @` opens the cite picker.
4. Validate every entry before you rely on them: `inkhaven sources check` (it exits non-zero on a problem, so it fits a CI step).
5. Send the bibliography back out for your reference manager or your typesetter: `inkhaven sources export --format csl-json --out sources.json` (or `--format bibtex`).

Stress a claim before a reviewer does

1. Test a claim against independent sources: `/triangulate the treatment reduced mortality by a third`. The verdict is whether the sources *concur*, not whether one asserts it.
2. Turn the adversary on your argument: `Ctrl+B J`, then the `prosecutor` persona, which tries to break the claim; the `defender` answers it.
3. Audit the whole Facts book for accuracy and for contradictions between claims: `/factcheck`. Check your declared axioms for internal coherence: `/undisputed`.
4. Catch a reference that has quietly died before a referee's click does: `/deadsources`.

WHAT YOU LEARNED

- Scientific writing is **nonfiction held to a hostile standard**: every load-bearing claim sourced, the argument built to survive a reviewer. Set `genre: "science"` (or `academic`) — and read the nonfiction chapter first.
- **Gather** from the literature with the Research Assistant's scholarly reach (DOI, arXiv), which auto-files citations; **triangulate** and **refute** so a claim is concurred and stress-tested, not merely stated.
- **Cite** through the **Sources** book — `sources import/export` (BibTeX, CSL-JSON) round-trips with Zotero, `sources check` fits CI, the cite picker (`Ctrl+V @`) drops references inline, and `/deadsources` catches link-rot.
- **Read adversarially** with the `prosecutor` / `defender` verdict personas and the `expert-reviewer`; verify with `/factcheck`, `/undisputed`, and computed `/calc` facts — find the hole yourself before the referee does.

CHAPTER 10

10

The theology & philosophy track

Theology and philosophy argue about things measurement cannot settle. You are not claiming a fact the reader can check against the world; you are advancing a *position* and asking that it be granted a hearing. The track's obligations are therefore unusual: your ground is *internal coherence* and the *tradition you answer to*, and your risks are not being wrong about a fact but being incoherent — or arguing against a straw version of the view you oppose. This chapter is the loop for the essay, the treatise, the sermon, the philosophical argument.

Frame — the genre that stops demanding proof

Set the genre, and this time the setting does something the other tracks never need:

EDIT · inkhaven.hjson

```
genre: "philosophy"
```

`theology`, `theological`, and `religious` share a nearby frame. What this genre does is *tell the AI readers to stop asking for empirical proof.* Point a naive fact-checker at "the soul is immortal" and it will flag an unsupported claim — which is not a defect in your argument but a category error in the reader. Declaring the genre recalibrates the interrogators: they stop asking "is this true?" and start asking "does this *hold together*, and have you met the strongest form of the objection?" — which are the only questions this kind of writing can be held to.

INSIGHT

Every track tunes its readers, but this is the one where the tuning is load-bearing. Without it, the tools apply the wrong standard and generate noise; with it, they apply the standard the work was actually written to. On this track, setting the genre is not a nicety — it is the difference between a useful reader and a confused one.

Gather — the tradition you answer to

Philosophy and theology are conversations centuries deep; a position that ignores the tradition it stands in is weaker for it. So the gathering here is *the reading*: use the Research Assistant to build a corpus of the thinkers and texts you engage, each kept with its provenance, so a claim about what a source argued can be checked against the source. And where your argument turns on recurring symbols or a named tradition — a theological reading that invokes a particular lineage — the Mythology book lets you *declare* the traditions you are working within, so the readers honour your frame rather than importing another.

Read — the moral reader and the coherent argument

This track has a reader of its own. The **Inner Theologian** (`Ctrl+B J`, then `T`) reads a passage for its moral and theological weight — the consequence left undepicted, the claim that carries more freight than the prose admits — through the lens of the tradition rather than against a fact. It is the questioner built for exactly this kind of writing.

Alongside it, the Inner Socrates roster offers the `philosophical-reader` and `theological-reader` personas, which grant your frame and press your reasoning: where is the hidden premise, what does this argument presuppose, which objection have you not met? Because the genre is set, even the general interrogator reads your prose as an argument to be tested rather than a set of facts to be verified.

Coherence over correctness

The deterministic check that fits this track is not the fact-checker but its cousin for *internal* consistency. Declare your axioms — the positions you take as given — and `/undisputed` checks them for coherence with one another: not "is this true in the world?" but "do these commitments contradict each other?" It is the mechanical half of the discipline the whole track is about — an argument that holds together on its own terms.

> TERM **Internal coherence**
>
> Consistency judged *within* a frame rather than against the external world — the standard by which an argument about the unmeasurable is fairly assessed. A theology may be coherent or incoherent regardless of whether its claims are empirically testable; on this track, coherence is the thing the tools check and the thing a fair opponent presses, precisely because correctness is not on the table.

> **PITFALL**
>
> The characteristic failure of the track is the straw man — defeating a weak version of the view you oppose and mistaking it for a victory. Use the Socratic readers deliberately against yourself here: ask them to state the *strongest* form of the objection you are answering, and to find where your argument only defeats a weaker one. An argument that beats the best opposing case is worth something; one that beats a caricature persuades no one who matters.

Produce

`export pdf|epub|docx` renders the essay or the treatise; if it cites — and serious philosophy and theology usually do — the Sources machinery of the scientific track is yours as well. The work leaves the desk as an argument that has already been pressed, in your own study, by the readers built to press it.

Hands-on: two procedures

Declare your axioms and check they cohere

1. In the Facts book, record the positions you take as given — the commitments your argument rests on.
2. Mark each as authorial rather than checkable: select it in the Facts tree and press `u` (the ※ glyph). An undisputed fact sits outside the trust ladder — true within your work by decree, with no external truth to test it against.
3. Check they hold together: `/undisputed` (or `inkhaven fact-check` on the undisputed set) asks not "is this true in the world?" but "do these commitments contradict each other?" — the mechanical half of coherence.

Read morally, and press your own argument

1. Scan the manuscript for moral and theological weight: `inkhaven theologian scan`. It reads each passage through the lens of the tradition rather than against a fact.
2. Open a deeper session on a passage: `inkhaven theologian session`, or `Ctrl+B J` then `T` in the editor.

3. Grant your frame and press the reasoning: `Ctrl+B J`, then the `philosophical-reader` or `theological-reader` persona. Ask it, deliberately, to state the *strongest* form of the objection you are answering — and to find where your argument only defeats a weaker one.

WHAT YOU LEARNED

- Theology and philosophy argue the **unmeasurable**: your ground is **internal coherence** and the **tradition** you answer to; your risks are incoherence and the straw man.
- Setting `genre: "philosophy"` (or `theology`) is **load-bearing** — it recalibrates the readers to stop demanding empirical proof and start pressing whether the argument holds.
- **Gather** the tradition as a research corpus, and **declare** the traditions you work within (Mythology) so the readers honour your frame.
- **Read** with the **Inner Theologian** (`Ctrl+B J` → `T`) and the `philosophical-reader` / `theological-reader` personas; check **coherence** with `/undisputed`; and turn the questioners against your own argument to defeat the strongest objection, not a caricature.

PART IV

Closing

CHAPTER 11

11

One desk, many books

You have now walked eight tracks, and if a pattern has surfaced it is this: they are not eight tools but one, held eight ways. The desk never changed. The tree of typed nodes, the two chord families, the companion books, the readers, the press — every track drew on the same apparatus. What changed was the *proportion*, and the proportion was decided by a single question: what must this book stay true to?

The four faculties, once more

Look back and the four faculties from the introduction are visible under every guide. **Structure** carried the technical manual and the scenario, where findability *is* the

product; it mattered less to the lyric novel. **Grounding** was a compiled world for fiction, a corpus of verified fact for nonfiction, a specification for documentation, the literature for science — the same faculty, four different things to be true to. **Reading** turned a different cast of questioners on each: the attentive fan, the system architect, the hostile referee, the good-faith opponent. And **production** was the same press throughout, scoped by the same status marks.

Learn to see a book as a balance of these four, and you will place any project — including ones no track here names — in an afternoon. A cookbook is nonfiction whose structure matters like a manual's. A work of narrative history is nonfiction that gathers a world the way fiction does. A philosophical novel is two tracks at once, and you already know both.

Most books are more than one track

The tracks were laid out as pure types so their differences would be visible. Real books are almost never pure. A hard-SF novel is fiction and science fiction and, where its future society is on trial, utopia. A popular-science book is scientific in its sourcing and nonfiction in its shape. A theological essay grounded in scholarship is two chapters of this book at once.

This is why the guides were written to combine rather than to exclude. When your book straddles, read both guides and take the tools that earn their place from each — the world simulation from fiction *and* the Sources book from science, the coherence checker from utopia *and* the technology ledger from science fiction. Nothing forbids it. The genre line picks the frame that dominates; the tools are all still there for the borrowing.

> **INSIGHT**
>
> The track is a starting point, not a verdict. You choose one to focus your first week — to know which of the many tools are yours — and then you let the book tell you what it actually needs. A novel that discovers it wants a bibliography grows one; a manual that discovers it wants a world builds one. The tool bends to the work. That was always the point.

The loop was the constant

Every track ran the same loop: frame the book, gather what grounds it, draft against that ground, read and revise, and return to gather more where the draft exposed a gap. It is worth saying plainly, because it is the one habit that outlasts any particular tool: *do not write into a void, and do not revise on faith.* Gather first, so the page has something to stand on. Read after, so the revision answers to something real. Between those two disciplines, everything Inkhaven offers is an amplifier — and outside them, no tool helps at all.

A last word on the tool that gets out of the way

The best thing a writing tool can do is disappear. Everything in this book — the compiled world, the graded facts, the questioning readers, the controlled glossary — exists so that a specific worry can leave your mind and live in the tool instead, freeing you to do the one thing no software can: write the sentence. The world will not contradict itself, so you need not hold it all in your head. The fact carries its source, so you need not remember where you read it. The reader will ask the hard question, so you need not pretend you already know it.

Pick your track. Set the genre. Gather what your book must be true to. Then close the panels, and write.

WHAT YOU LEARNED

- The eight tracks are **one apparatus held eight ways** — what changes is the **proportion** of the four faculties, decided by what the book must stay true to.
- Learn to read a book as a balance of **structure, grounding, reading, and production**, and you can place any project — even ones no track here names.
- **Most real books mix tracks**: read the guides that apply and borrow the tools that earn their place from each; the genre picks the dominant frame, the tools are all still available.
- The constant across every track is the **loop** — gather before you draft, read after you revise — and the goal is always the same: a tool that gets out of the way so you can write the sentence.

APPENDIX A

A

The keybinding reference

This appendix is the complete map of the desk — every chord Inkhaven's editor answers to, grouped by where it lives, with a word on what each does. You do not need to memorise it; press `Ctrl+B H` at any time for the same table inside the tool, always current for your version and your customisations. Keep this appendix for the mornings when you know there is a faster way and can't remember the keys.

How the chords are organised

Most keys fall into one of two families, reached through a *prefix* you press and release before the next key. Learn the two doors and the whole tool is two keystrokes away.

Ctrl+B … — The meta chords — the machinery around the manuscript: the companion books, the world, the readers, building, and project-wide actions.

Ctrl+V … — The view chords — the readers and the connective tissue: links, citations, the Inner Editor, timelines, threads, and structure views.

Ctrl+Z … — The Bund chords — scripting: run a script, open a shell pane, evaluate an expression against the embedded VM.

A handful of the most-used actions have no prefix at all (below), and some chords change meaning by *scope* — whether your focus is in the Editor or in the Tree — which is noted where it matters.

> **NOTE**
>
> Every binding is rebindable. The chord table lives in a config Inkhaven reads at startup (and scripts can change with `ink.key.*`), so if a chord fights the muscle memory you brought from another editor, change it rather than fighting it. The chords printed here are the defaults.

Core navigation — no prefix

Ctrl+Q — Quit, saving first.

Ctrl+S — Save the current paragraph.

Ctrl+1 — Focus the Editor — where you write.

Ctrl+2 — Open the Outline — the whole book as a jumpable map.

Ctrl+3 — Focus the AI chat pane, grounded on your book.

Ctrl+4 / Ctrl+/	Focus Search — full-text and semantic across the project.
Ctrl+T	Focus the Tree — the structure on the left.
Ctrl+I	The AI prompt line — a quick instruction without leaving the editor.
Alt+← / Alt+→	Back / forward through navigation history, like a browser.

Editing — no prefix

Ctrl+A	Select all.
Ctrl+C / Ctrl+K / Ctrl+P	Copy / cut / paste.
Ctrl+U / Ctrl+Y	Undo / redo.
Ctrl+D	Delete the current line.
Ctrl+E / Ctrl+W	Delete to end / to start of line.
Ctrl+F / Ctrl+X / Ctrl+R	Find / find-next / replace within the paragraph.
F3	Load the paragraph into the working buffer.
F4 / Ctrl+F4	Split-edit: open a second version of the paragraph / accept it.
F5	Take a snapshot of the paragraph (same as Ctrl+B N).
Ctrl+H / Ctrl+J	Scroll the lower split-edit pane.

Meta chords — `Ctrl+B`, in the Tree

With focus in the Tree, the meta chords reshape structure.

Ctrl+B c / s / p	Add a chapter / subchapter / paragraph.
Ctrl+B d	Delete the selected node.
Ctrl+B m	Morph the node to another type.

Ctrl+B ↑ / ↓ (u / j)	Reorder the node up / down among its siblings.
Ctrl+B q	Imposition preview — how the pages will fall on a printed sheet.

Meta chords — `Ctrl+B`, in the Editor

These reach the companion books and the per-paragraph tools.

The companion-book lookups

Ctrl+B p	Places — locations and settings (or insert an image).
Ctrl+B c	Characters — the cast.
Ctrl+B g	Notes — free-form working notes.
Ctrl+B y	Artefacts — objects and items.

The paragraph tools

Ctrl+B n	Snapshot the paragraph.
Ctrl+B r	Cycle the paragraph's status (napkin → … → ready).
Ctrl+B t	Rename the node to its first sentence.
Ctrl+B f	The function picker — a menu of paragraph actions.
Ctrl+B s	Read the paragraph aloud (text-to-speech).
Ctrl+B q / Shift+Q	Translate the paragraph into / out of an invented language.
Ctrl+B d	Deterministic rule-translate the paragraph to the Output pane.
Ctrl+B Shift+X	Fact-check the paragraph against the world and the facts.
Ctrl+B Shift+S	Search the Facts book.
Ctrl+B Shift+J	Jump to the next fact finding.
Ctrl+B Shift+H / Shift+M	Sentence-rhythm gauge / AI rhythm rewrite.

Ctrl+B Shift+T	Show-don't-tell scan of the paragraph.
Ctrl+B Shift+F / Shift+K	Style-warning overlay / echo (repeated-word) overlay.
Ctrl+B Shift+R / Shift+E	Save the paragraph as audio / reader-pace estimate.
Ctrl+B < / >	Jump to the previous / next scene break.

Meta chords — `Ctrl+B`, project-wide

Ctrl+B h	The quick-reference overlay — this table, live.
Ctrl+B i / v	Book info / credits.
Ctrl+B l	The LLM provider picker.
Ctrl+B a / b	Assemble / build the book.
Ctrl+B Shift+B	Back up the whole project to an archive.
Ctrl+B Shift+C	The unified review pass — readers and checks in one sweep.
Ctrl+B $	The cost dashboard — what the AI features have spent.
Ctrl+B w / Shift+W	The World overview / typewriter (focus) mode.
Ctrl+B j	The Inner Socrates reading overview.
Ctrl+B k	The AI pane, full-screen.
Ctrl+B x	The ConLang hub — constructed languages.
Ctrl+B Shift+O	The Outline, project-wide.
Ctrl+B 1–7	Filter the tree by paragraph status.
Ctrl+B] / }	Tag the paragraph / search by tag.
Ctrl+B 0 / Shift+0	Edit the HJSON config / open the doctor panel.

Ctrl+B Shift+V	The text-to-speech voice picker.
Ctrl+B Shift+G	The writing-streak heatmap.
Ctrl+B Shift+L	The concordance — every term and where it appears.
Ctrl+B Shift+P / Shift+N	The point-of-view chip / prompt-language mode.
Ctrl+B e / o / u	Toggle sound / take a 'take' / undo a deletion.

View chords — `Ctrl+V`

The view chords open the readers and the web of connections between nodes.

The readers and craft passes

Ctrl+V o	The Inner Editor overview — craft attention on the draft.
Ctrl+V v / Shift+V	Prose voice check / ambient voice mode.
Ctrl+V Shift+R	An editorial pass over the current stretch.
Ctrl+V y	A style-transfer rewrite of the selection.
Ctrl+V Space	The command palette — search for a chord you've forgotten.

Links, citations, and reuse

Ctrl+V @	The cite picker — drop a citation where the cursor is.
Ctrl+V a / i	Add a link / show incoming links to this node.
Ctrl+V l / k	List this node's links / backlinks.
Ctrl+V t	Open the link target under the cursor.
Ctrl+V b / m	Bookmark this node / list bookmarks.
Ctrl+V x / Shift+X	Insert a snippet / the snippet overview.
Ctrl+V z / Shift+Z	The terminology overlay / declare an intent.

Ctrl+V p / Shift+P	The paragraph picker / recent paragraphs.
Ctrl+V Shift+B	Look up a sibling book in a split pane.

Structure, world, and timeline

Ctrl+V g	Progress toward goals (then e to edit goals).
Ctrl+V Shift+K / Shift+L	The plan outline / the story bible.
Ctrl+V w / Shift+W	The story graph for the paragraph / the whole book.
Ctrl+V Shift+N / Shift+M	The character-arc view / the myth heatmap.
Ctrl+V Shift+F	A deep world refresh.
Ctrl+V e / Shift+E	The event picker / a new event.
Ctrl+V Shift+T	The timeline swim-lane.
Ctrl+V Shift+H	The threads picker — setups and payoffs.
Ctrl+V Shift+A / Shift+D	An AI thread audit / the thread doctor.

Submissions, comments, and misc

Ctrl+V u / q	The submissions tracker / generate a submission package.
Ctrl+V Shift+C	The comments panel.
Ctrl+V Shift+J	The journal / intelligence dashboard.
Ctrl+V Shift+U	The kill-ring picker (recent cuts).
Ctrl+V 1 / 2	Export the paragraph / subchapter as a Markdown buffer.
Ctrl+V s	Similar-mode — find passages like this one.

Bund chords — `Ctrl+Z`

For scripting the tool itself in the embedded Bund language.

Ctrl+Z r / n	Run the script buffer / start a new script.
Ctrl+Z e / ?	The eval modal / the script picker.
Ctrl+Z o / Shift+O	The Nushell pane / full Nushell.
Ctrl+Z h	Run the shell on the selection.
Ctrl+Z p / f	A haiku / full-screen the right pane.

A word on scope and discovery

Two ideas make the whole set learnable. First, *scope*: a bare letter after `Ctrl+B` often does different things in the Tree than in the Editor — `c` adds a chapter in the Tree but opens Characters in the Editor — because the tool knows what you are looking at. Second, *discovery*: you never have to remember any of this cold. `Ctrl+B H` prints the live table, and `Ctrl+V Space` opens a searchable palette where you type what you want and let Inkhaven find the chord. Learn those two, and every other key in this appendix is something you look up once and then, quietly, stop needing to.

APPENDIX B

B

The command reference

Every command this book put to work, gathered in one place and grouped by the part of the process it serves. This is a reference, not a tutorial — for the full treatment of any subsystem, the sibling books go deep; here you have the whole surface at a glance, so you can find the command you half-remember. Add `--help` to any command for its exact flags.

Starting and shaping a project

- `inkhaven init <path> [--template T]` — create a project. Templates: `empty`, `novel`, `nonfiction`, `technical`, `rpg-sourcebook`, `nanowrimo`.

- `inkhaven tui` — launch the full editor. Where most of the work happens.
- `inkhaven add <book|chapter|subchapter|paragraph> <title>` — add a node.
- `inkhaven list` — print the project tree; `inkhaven outline` — an indented outline.
- `inkhaven mv <path> up|down` — reorder a node; `inkhaven delete <path> --yes` — remove one.
- `inkhaven paragraph copy|move` — move a paragraph across parents.
- `inkhaven search <query>` — full-text and semantic search; `inkhaven stats` — per-paragraph stats.
- `inkhaven reindex [--prune] [--adopt]` — re-index hand-edited files back into the store.

Worldbuilding

The world model

- `inkhaven realworld new "<name>"` — create a `world.hjson` world definition.
- `inkhaven realworld validate` — check the definition is well-formed.
- `inkhaven realworld compile [--materialize]` — grow the world; `--materialize` writes it into the World book.
- `inkhaven realworld proposals` — list what the world offers (settlements, calendar, history); `… accept <id>` to take one.
- `inkhaven realworld propose-myth | propose-rulers | propose-language` — AI proposals seeded from the world.

Reading the world into the manuscript

- `inkhaven realworld scene --place <name> --day <N>` — a scene brief: season, weather, biome, nearest feature.
- `inkhaven realworld weather --day <N> --lat <deg>` — local weather at a day and latitude.
- `inkhaven realworld travel --from <p> --to <p>` — is a journey plausible for its distance and mode?
- `inkhaven realworld places` — the Place ↔ world cross-references.
- `inkhaven realworld set-coords <name> --lat <d> --lon <d>` — position a Place on the grid so the map draws it.

- `inkhaven realworld gazetteer [--output <f>]` — a consolidated Markdown world reference.
- `inkhaven realworld map` — render a map with the external plakat tool.

The world's people, past, and checks

- `inkhaven realworld history | chronicle` — the world's history, as events / as a state trajectory.
- `inkhaven realworld polities | culture | trade | ecology` — nations, cultures, trade routes, and life.
- `inkhaven realworld name` — propose settlement names in each realm's style.
- `inkhaven realworld co-location` — flag a character in two places at overlapping times.
- `inkhaven realworld coherence <node>` — an LLM pass over a container for cross-paragraph contradictions.
- `inkhaven realworld critique` — an AI reading of the whole world for consistency and realism.
- `inkhaven world utopia-check` — read the declared premises as a chain of claims and find where the society cheats.
- `inkhaven world utopia-suppress <id>` — mark a tension as intended; `inkhaven world utopia-refresh` — re-check.
- `inkhaven character arc | check | refresh | plan` — declare and test character arcs and agency.
- `inkhaven myth scan | check | profile | refresh | suppress` — the declared symbol / motif / archetype layer.
- `inkhaven event add | list | show | critique` — the story timeline; `inkhaven export-timeline [typst|svg|png]`.

Research, grounding, and continuity

- `inkhaven research` — the Research Assistant. Inside it, slash-commands do the work: `/fact`, `/note`, `/web`, `/wikidata`, `/geonames`, `/openalex`, `/arxiv`, `/gutenberg`, `/triangulate`, `/factcheck`, `/undisputed`, `/synthesize`, `/outline`, `/gaps`, `/upgrade`, `/stale`, `/deadsources`, `/sources`, `/calc`, `/world`, `/rag`, `/chain`.
- `inkhaven facts init --genre G` — seed a Facts book's continuity categories (`general`, `fantasy`, `scifi`, `mystery`, `historical`).
- `inkhaven facts scan | check | list | import | extract` — find, audit, and manage continuity facts.

- `inkhaven fact-check` — audit the manuscript against the world and the kept facts.
- `inkhaven continuity extract | list` — the continuity bible.
- `inkhaven drift scan | list` — style and continuity drift across the book.
- `inkhaven terms suggest | check` — propose and enforce canonical terminology.
- `inkhaven thread doctor | add | list | export` — plot threads: setups, developments, and payoffs.

Citations and sources

- `inkhaven sources list` — every citation in the Sources book.
- `inkhaven sources import <file.bib>` — bring citations in from BibTeX (e.g. Zotero).
- `inkhaven sources export --format bibtex|csl-json [--out <f>]` — write citations out.
- `inkhaven sources check` — validate entries; exits non-zero on a problem (fits CI).

Constructed languages

The ConLang suite is large (its own book, *Constructed Language Development*, covers it in full). The families, each `inkhaven language <action> <lang>`:

- **Lexicon** — `init`, `add-word`, `import`, `generate-lexicon`, `dictionary`, `gaps`, `audit`, `doctor`, `query`.
- **Phonology** — `generate-word`, `syllabify`, `ipa`, `stress`, `romanize`, `tone`, `sound-change`, `reconstruct`.
- **Grammar** — `grammar`, `define-rule`, `derive`, `gloss`, `paradigm`, `sentence`, `agree`, `grammar-book`.
- **Translation** — `translate`, `reverse`, `cross`, `remember`, `memory`, `corpus`, `eval`, `export-translation`.
- **Varieties & contact** — `varieties`, `lect`, `dialects`, `borrow`, `areal`, `cognates`, `family-tree`.
- **Writing systems** — `font-build`, `glyph-draft`, `glyph-lint`, `transliterate`, `spatial-typst`.

- **World links** — `link-place`, `link-character`, `speakers`, `ecology`, `idiolect`; and `export` to a portable format.

The AI readers

- `inkhaven inner-socrates check | persona | findings | ledger | suggestions` — the Socratic interrogator and its personas.
- `inkhaven inner-editor engage | findings | intent | suggestions | config` — the craft reader.
- `inkhaven theologian scan | session | suppress` — the moral / theological reader.
- `inkhaven show-dont-tell …`, `inkhaven prose profile|refresh|suggest`, `inkhaven dialogue …`, `inkhaven editorial` — the craft passes.

Producing the book

- `inkhaven build` — assemble the chapters and compile (the headless twin of `Ctrl+B B`).
- `inkhaven export typst|pdf|epub|docx [--status S] [--tag T]` — render the manuscript, scoped; companion books are never exported.
- `inkhaven pdf …` — a finishing workshop: `impose`, `booklet`, `cover`, `barcode`, `watermark`, `preflight`, `merge`, `split`, and more.
- `inkhaven submission …` / `inkhaven submissions …` — generate a synopsis / logline / comparables, and track where the book has gone.

Housekeeping

- `inkhaven backup [--out <f>]` / `inkhaven restore <archive> --to <dir>` — archive and recover a project.
- `inkhaven doctor [--scan] [--autofix]` — health check and repair.
- `inkhaven cost` — what the AI features have spent; `inkhaven config` — read or set configuration.

- `inkhaven ai "<prompt>"` — a one-shot LLM inference; `inkhaven bund "<code>"` — evaluate a Bund expression.

> **NOTE**
>
> This is the surface the book touches, not the whole of Inkhaven — there is more, and `inkhaven --help` (and `inkhaven <command> --help`) is the always-current authority for your version. When a command here feels thin, that is the sign to open the sibling book that treats its subsystem in full.

AFTERWORD

About the author

Vladimir Ulogov.

Vladimir Ulogov has spent decades building infrastructure for distributed systems — the kind of software that watches other software. Early in his career he worked on monitoring and telemetry platforms; later years took him into federated observability, telemetry buses, and the architecture of systems that have to make sense of millions of data points without losing the thread.

Observability, in the end, is a discipline of **coherence** — of never reporting a state the system cannot account for, of insisting that every signal follow from something real. It is not an accident that a tool he built for writers carries the same instinct into every kind of book it helps make.

What makes him slightly unusual in his corner of the industry is a tendency to write his own tools — not in the sense of small utilities, but in the sense of programming languages. The Bund language (its compiler, its VM, its document store, its parser) lives in a long series of Rust crates on crates.io. `rust_dynamic`, `rust_multistack`, `rust_multistackvm`, `bundcore`, `zbus_universal_data_gateway` — each is a building block that exists because the off-the-shelf options didn't fit the shape of the work.

Why one tool for so many kinds of book

Inkhaven is Vladimir's personal reflection on how a literary tool can help the people who write books — *all* of them, not one favoured kind. A novelist, a technical writer, a research scientist, and a philosopher share more than they usually admit: each is trying to hold a large structure in their head without it quietly contradicting itself, and each is doing it in scattered tools that don't talk to one another. The map is in one program, the citations in another, the outline in a third, and the manuscript nowhere near any of them.

Inkhaven was built to close that gap for every one of them at once — to make the structure, the grounding, the reading, and the press a single room. This book exists because that generality has a cost: with so many tools present, a writer new to Inkhaven can't always see which ones are *theirs*. The tracks are the answer — a way of saying, plainly, here is the handful of tools your kind of book will actually use, and here is how they fit together.

A work of love

Inkhaven is open source. The licence is permissive — you can read it, fork it, study it, modify it, and pass it on. Strictly speaking, the licence also lets you sell it. The author would, gently but firmly, disagree with you doing that. Inkhaven was not designed as a *for sale* project. It is a work of love made for the authors who can least afford to pay for software, and turning it into a commercial product would betray the reason it exists. So please — don't.

It carries no analytics, no telemetry, no upsell. The binary will never phone home; the project will never have an "Enterprise" tier you have to escape from.

This was a deliberate choice. There are excellent commercial tools for writing. They cost money — which is fine for many writers, and a barrier for many more. Inkhaven exists for the second group: for the novelist building a trilogy on a battered laptop, for the game-master who shouldn't have to pick between rent and software, for the graduate student writing a thesis, for the engineer drafting in the same terminal where they already write code — for anyone who would benefit from a tool that respects their work without asking for a credit-card number.

A note on cooperation

Vladimir believes firmly in the human capacity for mutual help — that we make better work, and live better lives, when we share what we know and what we build. Open source is one of the most concrete expressions of cooperation our era has produced: code read, improved, and passed forward without payment, without permission, by people who will never meet.

If Inkhaven helps you finish your book — whatever kind of book it is — that is enough. If it gives you a chord pattern you adapt into your own tool, that's a gift back to the larger project of making software writers can love. As a society, we achieve the greatest things when we help each other rather than compete with each other.

Where to find more

`GitHub`	@vulogov — the source for Inkhaven, Bund, and the dozen-plus Rust crates that carry the infrastructure. Issues and PRs welcome.
`LinkedIn`	/in/vladimirulogov — posts on observability, the occasional long-form essay.
`YouTube`	@vulogov — talks and walkthroughs from the conference trail.
`X / Twitter`	@vladimir_ulogov

Whatever track you are on — if the tool ever gets in the way of the book instead of out of it, open an issue on GitHub. The author reads them.

END OF THE BOOK · INKHAVEN 1.6.10